PeopleDesk

Employee Management System

Project Walkthrough, Setup, and Run Guide

Frontend	HTML5, CSS3, JavaScript
Backend	Java, Spring Boot, Spring Data JPA, Maven
Database	MySQL
Version control	Git and GitHub
Frontend hosting	Netlify (backend/database hosting is separate)

Practical guide for running and explaining the project locally.

1. Project overview

PeopleDesk is a full-stack Employee Management System. It provides a dashboard and employee directory for viewing and managing employee records. The frontend sends HTTP requests to a Java Spring Boot REST API, and the backend reads or writes employee data in a MySQL database.

Main features

• Dashboard with employee statistics and charts.

• Employee directory with search, filtering, sorting, and pagination.

• Add, view, edit, and delete employee records.

• Individual employee profile page.

• CSV import and export.

• REST API for employee CRUD operations.

Technology stack

• HTML, CSS, JavaScript — browser interface.

• Java, Spring Boot — backend and REST API.

• Spring Data JPA — database access.

• MySQL — persistent employee storage.

• Maven — backend build and run tool.

• Git/GitHub — source control.

• Netlify — frontend hosting.

2. Project structure

Frontend files are in the repository root; the Spring Boot application is in the peopledesk subfolder.

```
employee-management-system/
■■■ index.html                # Dashboard
■■■ employees.html            # Employee directory
■■■ employee-profile.html     # Individual profile
■■■ script.js                 # Main frontend logic
■■■ employee-profile.js       # Profile logic
■■■ style.css                 # Shared styles
■■■ peopledesk/               # Spring Boot backend
    ■■■ pom.xml
    ■■■ mvnw / mvnw.cmd
    ■■■ src/
        ■■■ main/java/...     # App, entity, repository, controller
        ■■■ main/resources/
            ■■■ application.properties
```

Backup files such as index-backup.html, script-backup.js, and style-backup.css are not needed to run the application.

3. Requirements

• Install a Java JDK compatible with the version in peopledesk/pom.xml.

• Install and run MySQL Server.

• Use VS Code or another Java IDE.

• Use a browser and the VS Code Live Server extension.

4. Database setup

Start MySQL

Open MySQL Workbench and connect to your local MySQL Server. If peopledesk_db and the employees table already exist, keep using them; do not recreate or drop existing data.

Create the database if needed

```
CREATE DATABASE IF NOT EXISTS peopledesk_db;
```

The backend expects an employees table matching the Employee entity. Common fields include id, name, email, department, job_title, salary, status, photo_url, and created_at. Use the schema already working in your project.

Warning: Do not run destructive SQL commands such as DROP TABLE unless you intentionally want to delete data.

5. Configure database credentials safely

Use environment variables rather than committing real passwords to GitHub. The connection settings in application.properties should be similar to:

```
spring.datasource.url=${DB_URL:jdbc:mysql://localhost:3306/peopledesk_db}
spring.datasource.username=${DB_USERNAME}
spring.datasource.password=${DB_PASSWORD}
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
```

Do not put real passwords in committed files. If your properties file has fallback values after a colon, those values are visible in the file. Keep credentials private.

6. Run the backend

Open a terminal in VS Code

Open Terminal → New Terminal. From the main project folder, run:

```
cd C:\Users\viru\Documents\project\employee-management-system
cd peopledesk
```

Set environment variables in PowerShell

Replace these example values with your own MySQL credentials. Do not share them.

```
$env:DB_URL = "jdbc:mysql://localhost:3306/peopledesk_db"
$env:DB_USERNAME = "your_mysql_username"
$env:DB_PASSWORD = "your_mysql_password"
```

Start Spring Boot

```
.\mvnw.cmd spring-boot:run
```

Keep this terminal open. Wait for the successful startup message. The backend uses port 8080.

Test the API

Open this URL in a browser:

```
http://localhost:8080/api/employees
```

If the API and database are working, the browser should show JSON employee data. If it fails, check the first meaningful error in the backend terminal.

7. Run the frontend

• In VS Code Explorer, open index.html in the main employee-management-system folder.

• Right-click and choose Open with Live Server.

• The browser usually opens a URL similar to the one below.

```
http://127.0.0.1:5500/index.html
```

Use the exact URL shown by Live Server if it chooses a different port. Keep the backend and MySQL running.

8. How to use the application

Dashboard

Open index.html to view employee statistics and charts, then use navigation to open the directory.

Employee directory

Open Employees to search, filter, sort, and page through records. Use the available actions to add, view, edit, or delete employees.

Employee profile

Select View on an employee row. The profile page reads the employee ID from the URL and requests that record from the backend.

CSV import/export

Use Import CSV to select a CSV file with the expected employee columns. Check the import result afterward. Export CSV downloads employee data supported by the current frontend. Keep a backup before importing many records.

9. REST API reference

Method	Endpoint	Purpose
GET	/api/employees	List employees
GET	/api/employees/{id}	Get one employee
POST	/api/employees	Create employee
PUT	/api/employees/{id}	Update employee
DELETE	/api/employees/{id}	Delete employee

Base URL: http://localhost:8080

10. Troubleshooting

Employee data does not load

• Confirm MySQL is running.

• Confirm Spring Boot is running.

• Open http://localhost:8080/api/employees to test the API.

• Check browser Developer Tools → Console.

• Confirm the local frontend API URL points to http://localhost:8080/api/employees.

Backend cannot connect to MySQL

• Verify database name, username, password, and port.

• Set DB_URL, DB_USERNAME, and DB_PASSWORD in the same PowerShell terminal used to start Spring Boot.

• Confirm peopledesk_db and the employees table exist.

• Read the backend terminal's first meaningful error.

Port 8080 is already in use

Stop the other app using port 8080, or change server.port and update the frontend API URL to match.

Java or Maven fails

• Run java -version to check Java.

• Run the command from the peopledesk folder.

• Check Java compatibility in pom.xml.

• Use the complete terminal error to diagnose the problem.

11. Save future changes to GitHub

Run these commands from the main employee-management-system folder after making and testing changes:

```
git status
git add .
git commit -m "Describe your changes"
git push
```

Before committing, check that passwords, .env files, generated target files, and other secrets are not staged.

12. Deployment notes

The frontend is published on Netlify. The Spring Boot backend and MySQL database still need online hosting for public visitors to use employee data features. A public website's localhost URL points to each visitor's own computer, not your computer.

After deploying the backend, update the frontend API base URL to the public backend URL and configure CORS appropriately. The browser frontend should call the backend API; it should never connect directly to the database.

13. Quick-start checklist

• ■ MySQL Server is running.

• ■ peopledesk_db and employees table exist.

• ■ DB_URL, DB_USERNAME, and DB_PASSWORD are set in the backend terminal.

• ■ Backend started with .\mvnw.cmd spring-boot:run.

• ■ API responds at http://localhost:8080/api/employees.

• ■ Frontend started with Live Server.

• ■ Dashboard, directory, and profile pages tested.

GitHub repository: https://github.com/sgviru18/PeopleDesk